1 Pipeline and architecture diagrams

This section shows the control and data flow behind the algebra of the other chapters: the per-epoch estimation loop (Figure 1), the shared geometry kernel behind the three differencing regimes (Figure 2), the real-time SSR correction pipeline (Figure 3), the real-time streaming front-end that feeds it (Figure 4), and the joint estimator of the PPP-RTK network producer (Figure 5). Each box names the function or class that implements the step, so the diagrams also serve as a map into `src/` . The diagrams are maintained by hand and simplify the code: they show the main sequence, not every gate or diagnostic.

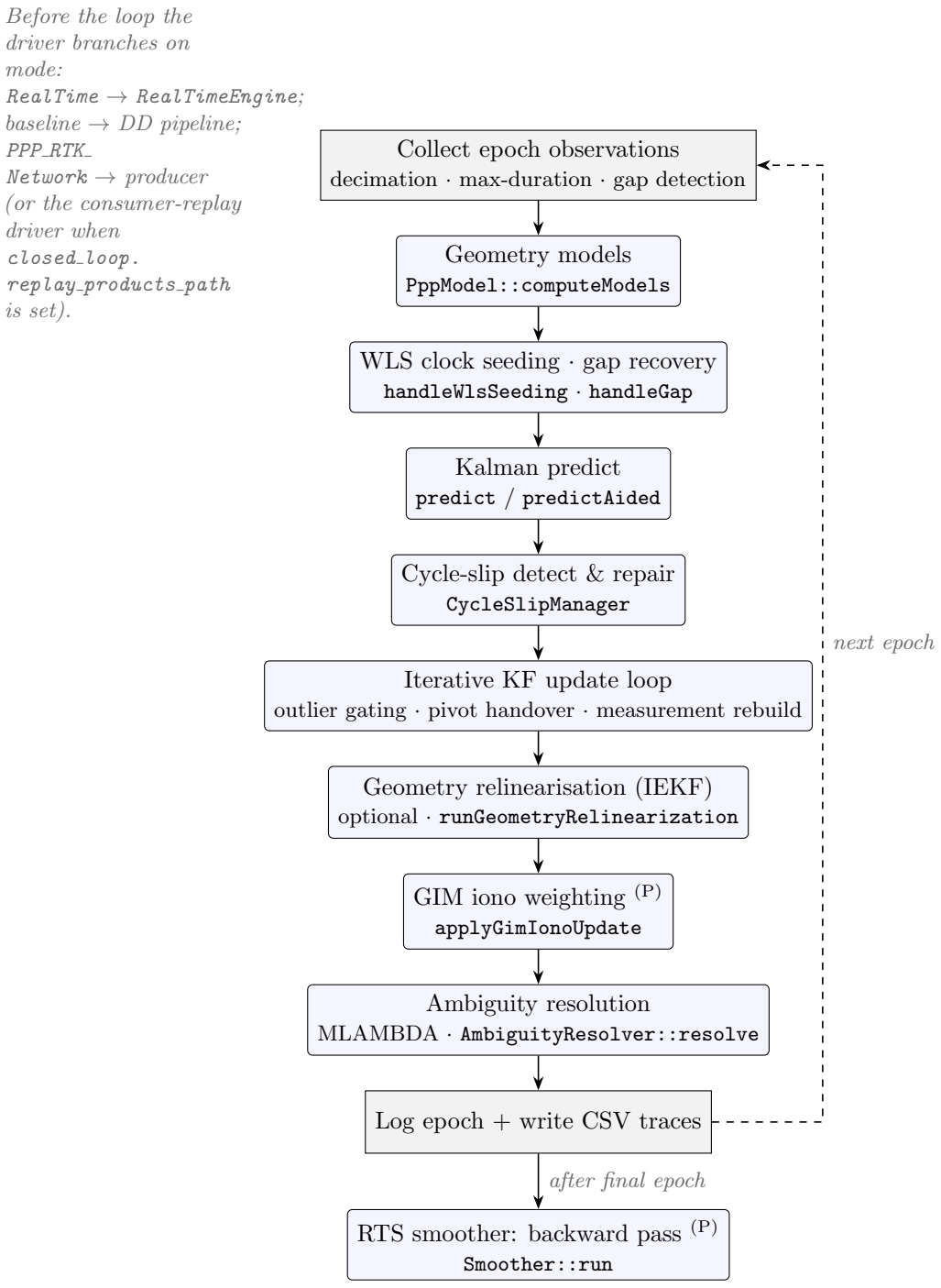


Figure 1: Per-epoch post-processing loop in `Application::run`. Solid arrows give the sequence within an epoch; the dashed arrow is the feedback from one epoch to the next. Steps tagged (P) run in post-processing only; the real-time path (`RealTimeEngine`) has no GIM source and no smoother. The smoother runs once, after the forward pass, and only when `run_smoother` is set.

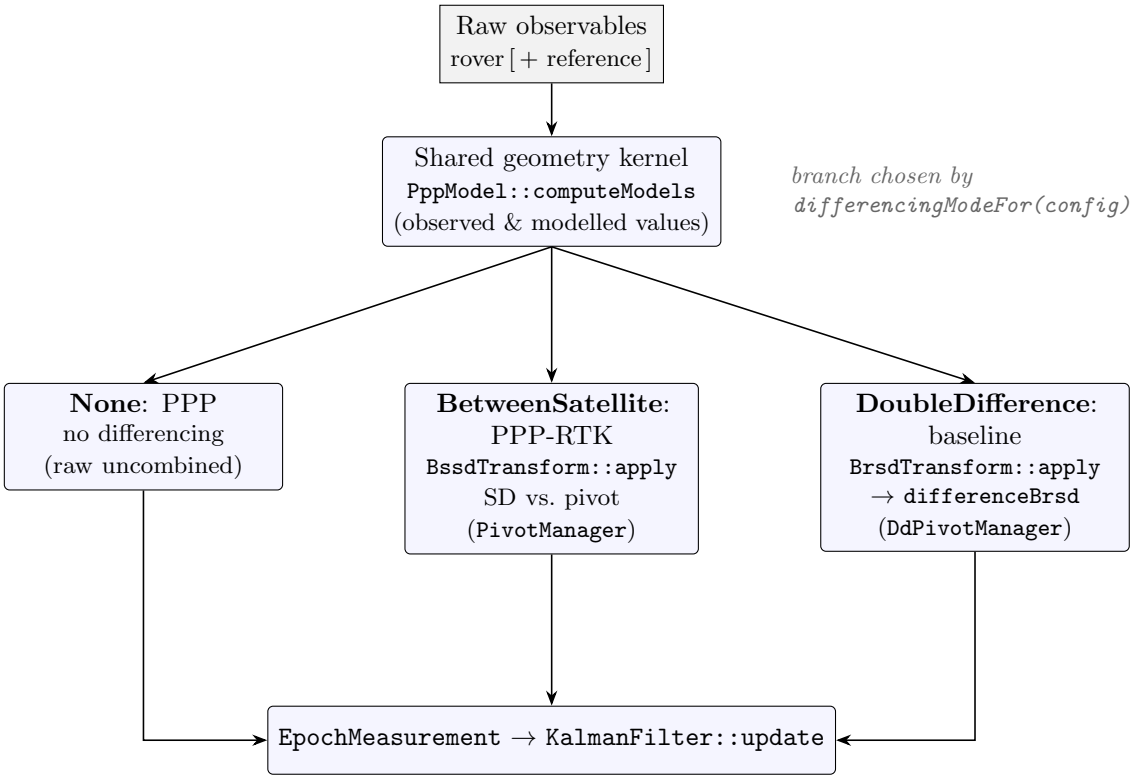


Figure 2: The three differencing regimes share one geometry kernel. The undifferenced observed and modelled values are computed once. The regime returned by `differencingModeFor` (selected once, when `MeasurementBuilderFactory` builds the measurement builder) then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline). All three feed the same Kalman update.

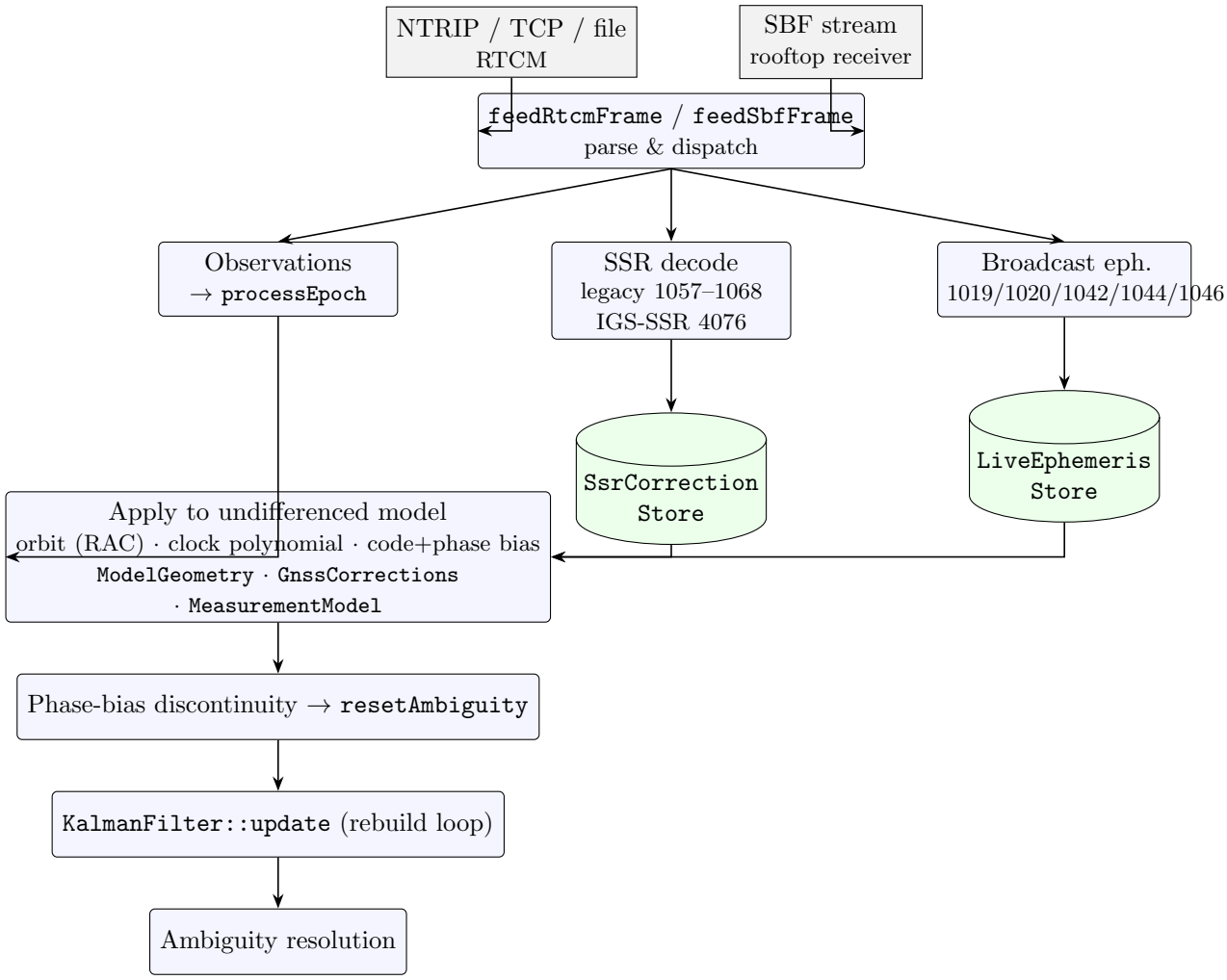


Figure 3: Real-time SSR correction pipeline. The decoder splits the byte stream three ways: observations go to `processEpoch`, SSR corrections to `SsrCorrectionStore`, and broadcast ephemerides to `LiveEphemerisStore`. The legacy SSR decoder accepts message types 1057–1068 and 1240–1270; the diagram lists the first range only. The undifferenced observation model then applies the precise orbit (RAC frame), the clock polynomial and the code and phase biases. Between the slip check and the Kalman update, `resetOnPhaseBiasDisc` reacts to a change of the phase-bias discontinuity counter, by default with an ambiguity reset; `reinflateOnCodeBiasDisc` and `reinflateOnClockDisc` follow for the code-bias and clock counters. See `RealTimeEngine::processEpoch`.

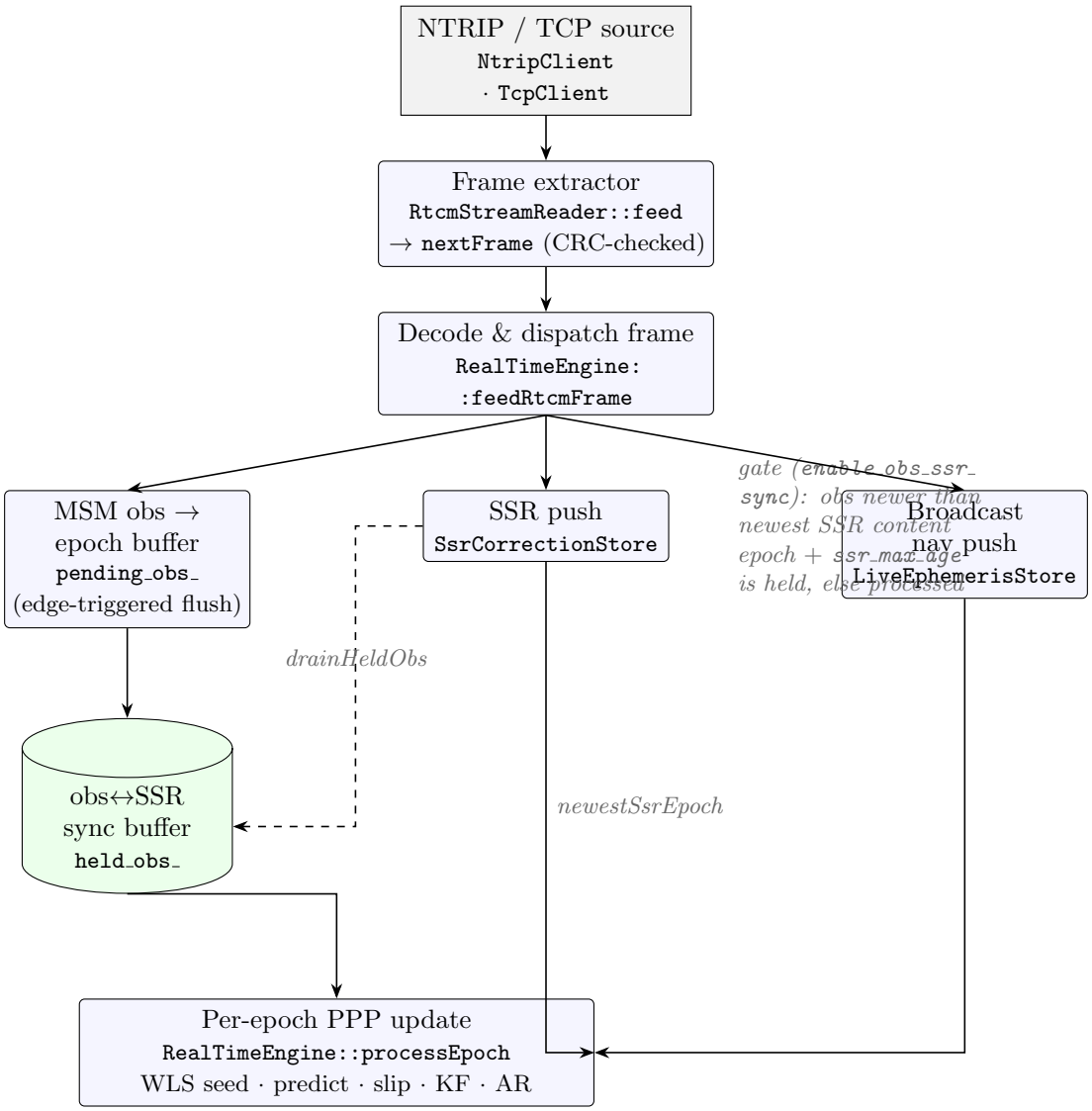


Figure 4: Real-time streaming front-end ahead of the SSR correction stage of Figure 3. `RtcmStreamReader` reassembles the raw bytes from `NtripClient` or `TcpClient` into CRC-validated RTCM3 frames, and `feedRtcmFrame` decodes and dispatches each frame. MSM observations accumulate in `pending_obs_`, which is submitted to `processEpoch` when a frame with a new epoch time arrives (the edge trigger). SSR corrections and broadcast ephemerides go to their stores. With `enable_obs_ssr_sync` set, the first admission gate of `processEpoch` (`admitBacklogAndSync`) holds an observation epoch that is newer than the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`). The held epoch is kept in `held_obs_` (only the latest one) and resubmitted by `drainHeldObs` after each SSR push. An epoch inside the horizon is processed at once. A hold longer than `obs_ssr_sync_max_hold_s` (default 30 s) latches a degraded outage state in which epochs pass through without SSR until the stream covers them again. The gate applies only with a live SSR orbit and clock source and never before the first epoch. See the real-time engine.

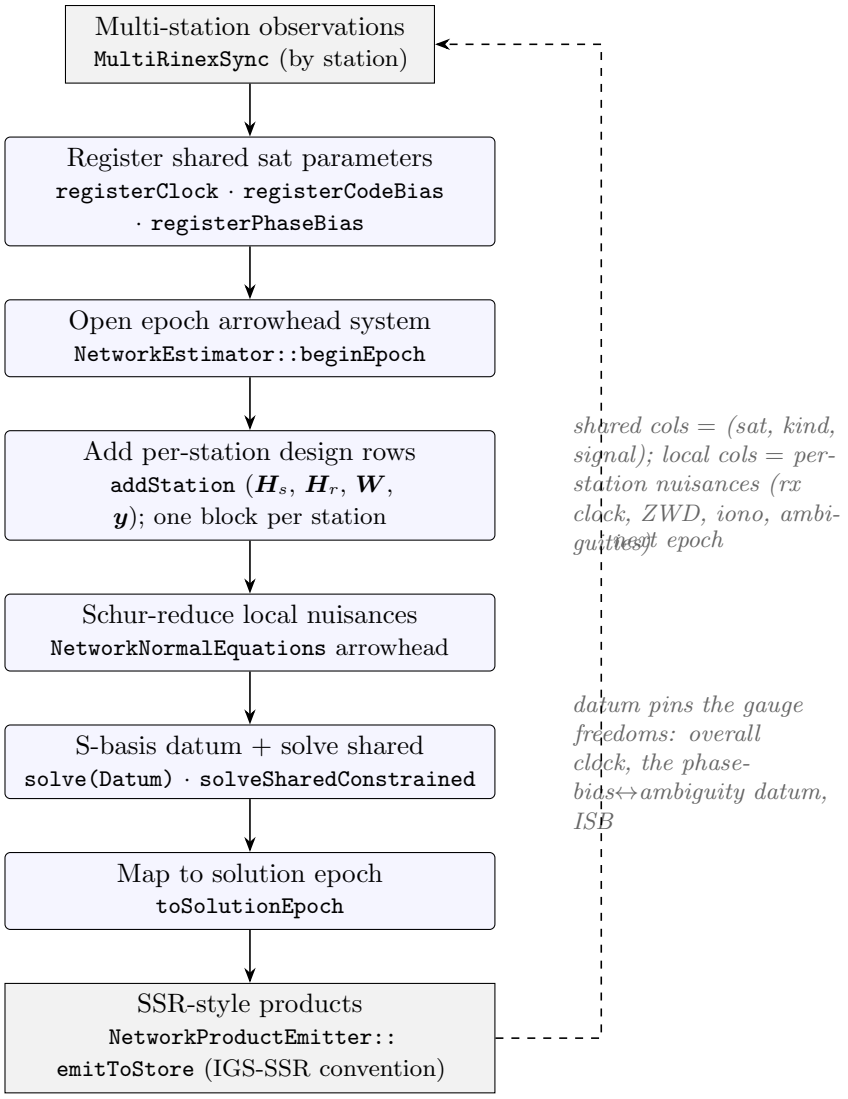


Figure 5: Joint estimator of the PPP-RTK network producer (`NetworkEstimator`). The shared satellite-side columns, one per (satellite, parameter kind, signal) for the estimable clock, code bias and phase bias, are registered afresh every epoch (`reset`, `registerClock`, `registerCodeBias`, `registerPhaseBias`, and `registerVtecCoeff` for the SH-VTEC coefficients when `network.estimate_iono_sh` is on); `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` adds one pre-formed block of local-versus-shared design rows per station, with no receiver-side differencing. Schur reduction removes the local nuisances; the reduced shared system is rank-deficient by the gauge freedoms. `solve` fixes them with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the estimate to a record that `NetworkProductEmitter` writes in the IGS-SSR convention. The driver `Application::runNetwork` runs this core once per epoch over the `MultiRinexSync` stations and only orchestrates: `networkPassAHead_` models each station, `networkAssemblyRegisterColumns_` registers the columns and calls `beginEpoch`, `networkPassB_` calls `addStation` (all in `app/ApplicationNetworkAssembly`), `networkSolveGlue` applies the datum and solves, `networkSolPrep` maps, and `networkEmitStore` emits. With `network.closed_loop.enable` set, the emit step also drives a held-out PPP-RTK consumer on the new products: BSSD by default, undifferenced with `closed_loop.consumer_undifferenced`. See the network mode and its producer-architecture section.